

Semiring-Based Linear Algebra Framework for Two-Way Regular Path Queries

Georgii Belianin
g.belianin@student.tue.nl
Eindhoven University of Technology
Eindhoven, The Netherlands

Iakov Kirilenko
y.kirilenko@spbu.ru
Saint Petersburg State University
Saint Petersburg, Russia

Rodion Suvorov
suvorov.53245324@gmail.com
Saint Petersburg State University
Saint Petersburg, Russia

Semyon Grigoriev
s.v.grigoriev@mail.spbu.ru
Saint Petersburg State University
Saint Petersburg, Russia

Abstract

Two-way regular path queries (2-RPQs) allow the use of regular languages over edges and their inverses in edge-labeled graphs to constrain paths of interest. Adopted in modern graph query languages such as GQL, SQL/PGQ, and Cypher, 2-RPQs have become part of the GQL ISO standard. However, their efficient evaluation on large-scale graphs remains challenging. In this paper, we propose a semiring-based generalization of the linear algebra approach for evaluating single-source 2-RPQs. The original LARPQ algorithm solves the reachability problem using Boolean matrices. We generalize it by replacing Boolean algebra with arbitrary semirings, enabling a unified solution for various path-aggregation problems. Specifically, we employ the path semiring to find all simple paths, all trails, and all shortest paths. The algorithms are implemented using the SuiteSparse:GraphBLAS library within the LAGraph infrastructure. Experimental evaluation on real-world and synthetic knowledge graphs shows that the approach is competitive with state-of-the-art graph database systems: it is slower on the typical simple query, but substantially faster on hard queries, which leads to lower total and mean evaluation times.

Keywords

Regular path queries, Two-way RPQ, Linear algebra, Semirings, Graph databases, Sparse matrices

1 Introduction

Language-constrained path querying (FLPQ) [6] provides a mechanism for searching paths in edge-labeled graphs where constraints are specified in terms of formal languages. A path is considered valid if the word formed by concatenating the labels of its edges belongs to the specified language. When constraints are regular languages, such queries are called regular path queries (RPQs). In practice, it is often useful to allow traversal opposite to edge directions, leading to two-way regular path queries (2-RPQs) [8, 11]. Queries specifying only a start or an end vertex are known as single-source or single-destination queries, respectively. Both have been adopted in graph query languages such as GQL [15], SQL/PGQ, and SPARQL [1].

Graph database systems support different path query semantics, as the choice significantly affects both result interpretation and computational complexity. Common semantics include:

EDBT '27, Lille (France)

© 2027 Copyright held by the owner/author(s). Published on xxxxx under ISBN xxx-x-xxxx-xxx-x, series ISSN xxxx-xxxx. Distribution of this paper is permitted under the terms of the Creative Commons license CC-by-nc-nd 4.0.

reachability (all vertices reachable via any valid path), all-shortest-paths (one shortest path per reachable vertex), all-simple-paths (all simple paths satisfying the query), and all-trails (all trails satisfying the query) [14]. Each semantics serves different use cases, and the query engine must select the appropriate algorithm accordingly.

Efficient evaluation of RPQs and 2-RPQs remains an active research area [12, 23]. Sparse linear algebra-based methods have shown promise for RPQ evaluation [4]. In particular, the LARPQ (Linear Algebra-based Regular Path Query) algorithm [7] employs a breadth-first traversal that simultaneously processes both the graph and the finite automaton representing the regular language. While LARPQ demonstrates promising performance, it was designed solely for reachability, and its application to path-related semantics (e.g., simple or shortest paths) remains unexplored.

The contributions of this paper are as follows.

- A formal generalization of the LARPQ algorithm to a semiring-based framework, enabling the solution of various path-aggregation problems within a unified algorithmic structure.
- Instantiation of the framework with specific semirings to solve distinct problems: Boolean semiring for reachability, and different path semirings with index unary operators for all-simple-paths and all-trails and all-shortest-paths semantics.
- Experimental evaluation on both real-world and synthetic knowledge graphs and queries, showing that the approach is competitive with modern graph database systems: it is slower on the typical simple query but substantially faster on hard queries, which yields lower total and mean evaluation times.

2 Preliminaries

In this section, we introduce the foundational concepts required throughout the paper, including edge-labeled graphs, paths, finite automata, and regular path queries.

2.1 Edge-Labeled Graphs

We begin by defining the graph data model used in this work.

Definition 2.1 (Edge-labeled graph). An edge-labeled graph (or simply a graph) is a quadruple $G = \langle V, E, L, \lambda_G \rangle$ where:

- V is a finite set of vertices;
- $E \subseteq V \times V$ is a finite set of edges;
- L is a finite set of labels;
- $\lambda_G : E \mapsto 2^L$ represents edge labels.

COMPRESS C1

Abstract: "However, their efficient evaluation on large-scale graphs remains challenging." is generic; cut. About 1 line.

COMPRESS C2

Merge this and the next sentence into one ("Single-source and single-destination variants ... have been adopted in GQL, SQL/PGQ and SPARQL"). About 1 line.

COMPRESS C3

Cut this generic sentence and the last one of the paragraph ("Each semantics serves ..."); keep only the enumeration of semantics, and fix its all-shortest-paths gloss, which describes ANY SHORTEST, not ALL SHORTEST. About 3 lines.

COMPRESS C4

Generic; fold into the next sentence ("Among the approaches to efficient (2-)RPQ evaluation [...], sparse linear algebra has shown promise [...]"). About 1 line.

COMPRESS C5

Contributions: bullet 2 can simply list the four instantiations; bullet 3 should be one line (the numbers belong to Sect. 6). About 3 lines. The introduction must also *gain* a short delta-vs-[6] paragraph (ADBIS D7, R4) and ideally a two-sentence motivating example (R3), so expect the section to stay about the same length.

For the remainder of this paper, we assume that vertices of any graph $G = \langle V, E, L, \lambda_G \rangle$ are enumerated, and without loss of generality, $V = \{1, 2, \dots, |V|\}$.

To enable traversal in both directions along edges, we introduce inverted labels and edges. Let $G = \langle V, E, L, \lambda_G \rangle$ be an edge-labeled graph. For every label $a \in L$ let $a^- \notin L$ be its inverse, with $(a^-)^- = a$ and $a = b \Leftrightarrow a^- = b^-$, and for every edge $e = (u, v)$ let $e^- = (v, u)$. The bidirectional graph $G^{\leftrightarrow} = \langle V, E^{\leftrightarrow}, L^{\leftrightarrow}, \lambda_G^{\leftrightarrow} \rangle$ is given by

$$E^{\leftrightarrow} = E \cup \{e^- \mid e \in E\}, \quad L^{\leftrightarrow} = L \cup \{a^- \mid a \in L\},$$

$$\lambda_G^{\leftrightarrow}(e) = \lambda_G(e) \cup \{a^- \mid a \in \lambda_G(e^-)\} \quad \text{for } e \in E^{\leftrightarrow},$$

where $\lambda_G(e) = \emptyset$ for $e \notin E$.

The graph $G^{\leftrightarrow}$ allows traversal along both original and inverted edges, which is essential for evaluating two-way regular path queries.

2.2 Paths

Definition 2.2 (Path). A path $\pi = (e_1, e_2, \dots, e_n)$ of length n in the graph $G = \langle V, E, L, \lambda_G \rangle$ from u_1 to v_n is a finite sequence of edges $e_i = (u_i, v_i) \in E$ such that $\forall 1 \leq i \leq n-1 : v_i = u_{i+1}$.

We denote $|\pi|$ as the length of a path π and consider zero-length paths, represented by an empty sequence from a vertex to itself.

Definition 2.3 (Simple path). A path $\pi = (e_1, e_2, \dots, e_n)$ is simple if it contains no repeated vertices. Formally, assuming $e_i = (v_i, u_i)$, this means that all vertices in the sequence $v_1, u_1, u_2, \dots, u_n$ are pairwise distinct.

Definition 2.4 (Trail). A path $\pi = (e_1, e_2, \dots, e_n)$ is a trail if it contains no repeated edges. Formally, $\forall 1 \leq i, j \leq n : e_i = e_j \Rightarrow i = j$.

Definition 2.5 (Two-way path). A two-way path (2-path) $\pi = (e_1, e_2, \dots, e_n)$ in G is a path in $G^{\leftrightarrow}$. Two-way simple paths and two-way trails are defined analogously.

The mapping $\omega_{G^{\leftrightarrow}}$ associates 2-paths with words over the label alphabet: $\omega_{G^{\leftrightarrow}}(\pi) = \{a_1 \cdot \dots \cdot a_n \mid a_i \in \lambda_G^{\leftrightarrow}(e_i)\}$ for a 2-path $\pi = (e_1, \dots, e_n)$ in G , where $\cdot$ denotes string concatenation.

2.3 Two-Way Finite Automata

Regular languages, recognized by finite automata, provide the foundation for specifying path constraints. For two-way path queries, we require a variant of nondeterministic finite automata.

Definition 2.6 (Two-way NFA). A two-way nondeterministic finite automaton (2-NFA) is a tuple $N = \langle Q, \Sigma, \Delta, \lambda_N, Q_S, Q_F \rangle$, where:

- Q is a finite set of states;
- Σ is a finite alphabet;
- $\Delta \subseteq Q \times Q$ is the transition relation;
- $\lambda_N : \Delta \mapsto 2^{\Sigma^{\leftrightarrow}}$ maps transitions to labels;
- $Q_S \subseteq Q$ is the set of starting states;
- $Q_F \subseteq Q$ is the set of final states.

A 2-NFA can be viewed as a graph $G_N = \langle Q, \Delta, \Sigma^{\leftrightarrow}, \lambda_N \rangle$ equipped with sets of starting and final states. Conversely, a graph $G = \langle V, E, L, \lambda_G \rangle$ equipped with $V_S \subseteq V$ can be seen as a 2-NFA $N_G = \langle V, L, E, \lambda_G^{\leftrightarrow}, V_S, V \rangle$ that accepts:

$$\llbracket N_G \rrbracket = \bigcup_{\text{2-path } \pi_G \text{ in } G \text{ from } v_S \in V_S} \omega_{G^{\leftrightarrow}}(\pi_G)$$

2.4 Two-Way Regular Path Queries

Definition 2.7 (Two-way regular path query). A two-way regular path query (2-RPQ) is a triple $\langle G, R, v_s \rangle$ where $G = \langle V, E, L, \lambda_G \rangle$ is a graph, R is a regular language over the alphabet $L^{\leftrightarrow}$, and $v_s \in V$ is a starting vertex.

We define the following semantic variants of 2-RPQs considered in this paper:

- Single-source all *simple* paths;
- Single-source all *trails*.
- Single-source *shortest* paths;

Let

$$\mathcal{A} = \{\pi \mid \pi \text{ is a 2-path in } G \text{ from } v_s$$

$$\text{and } \omega_{G^{\leftrightarrow}}(\pi) \cap R \neq \emptyset\}$$

be the set of 2-paths from v_s that satisfy the constraint. The results of the query are

$$\llbracket Q \rrbracket_{\text{simple}} = \{\pi \in \mathcal{A} \mid \pi \text{ is simple}\},$$

$$\llbracket Q \rrbracket_{\text{trails}} = \{\pi \in \mathcal{A} \mid \pi \text{ is a trail}\},$$

$$\llbracket Q \rrbracket_{\text{shortest}} = \{\pi \in \mathcal{A} \mid |\pi| \leq |\pi'| \text{ for every } \pi' \in \mathcal{A} \text{ ending in the same vertex as } \pi\}.$$

That is, under the shortest-paths semantics, for every vertex v all shortest paths among the 2-paths from v_s to v that satisfy R are returned, as in MillenniumDB [14, 23].

Single-destination variants reduce to the single-source ones by reversing the query. Let $R^- = \{a_n^- a_{n-1}^- \dots a_1^- \mid a_1 a_2 \dots a_n \in R\}$; then a path from u to v_s matching R is, read backwards in $G^{\leftrightarrow}$, a path from v_s to u matching R^- , so $\llbracket \langle G, R, v_s \rangle \rrbracket_{SD^*}$ is obtained from $\llbracket \langle G, R^-, v_s \rangle \rrbracket_{SS^*}$ by reversing the resulting paths. An automaton for R^- is obtained from an automaton for R by inverting transition directions, inverting labels, and swapping starting and final states; in matrix terms, the Boolean decomposition is transposed. For this reason, the rest of the paper considers single-source queries only.

Figure 1 presents examples of single-source query results under different semantics on the same graph: **a) all simple paths**, **b) all trails**, and **c) all shortest paths**.

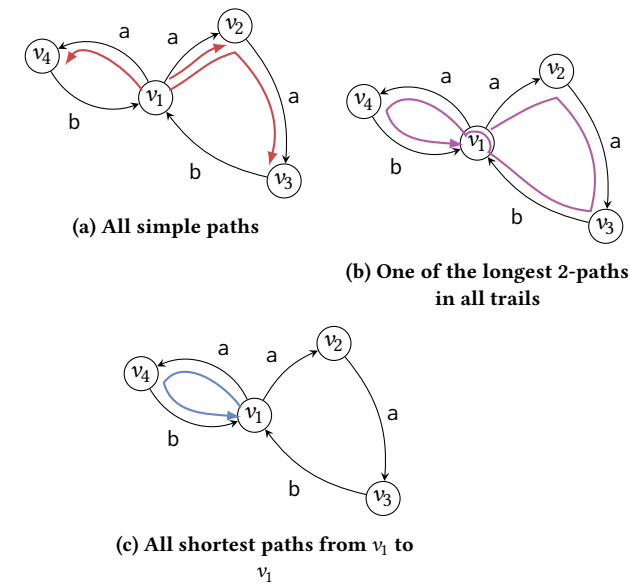


Figure 1: Different query semantics for $((a^+)(b^?))^+$

COMPRESS C7

Defs. 2.3/2.4: keep either the informal clause or the "Formally, ..." clause, not both. About 2 lines.

GB

Data model vs. MillenniumDB (checked in its code and in the PathFinder paper). Here an edge is a vertex pair with a set of labels, and e, e^- are distinct elements of $E^{\leftrightarrow}$. In MillenniumDB, RDF, and property graphs an edge is an identified object (for RDF, the triple): $\langle u, a, v \rangle$ and $\langle u, b, v \rangle$ are two edges, and an inverse traversal reuses the same edge (is_trail compares edge ids only, ignoring direction). Consequences: a path that uses two labels over the same pair is not a 2-trail here but a trail there; $u \rightarrow v \rightarrow u$ over one edge is a 2-trail here but not there; for all three semantics, label variants of one vertex sequence are one path here and several paths there (the implementation actually produces one copy per label, without storing the label). Simple paths are otherwise unaffected: "no repeated vertex" is the challenge version's semantics, called ACYCLIC in current MillenniumDB, whose SIMPLE allows first = last. Two ways out: (1) reimplement with labels stored in the paths, i.e., apply the extension per label a with l_a appending (a, j) , which makes trails and path counts coincide with MillenniumDB; or (2) keep the model, state the difference in Sects. 2 and 6, and show it is immaterial for the studied datasets by reporting, per query, whether result counts agree and how many results involve parallel matching labels or an inverse re-traversal. Decision needed before the evaluation is redone.

COMPRESS C10

The "graph as a 2-NFA" view (N_G and the displayed $\llbracket N_G \rrbracket$) is never used. Keep one sentence: "A 2-NFA is an edge-labeled graph over Q with labels from $\Sigma^{\leftrightarrow}$ ", so the matrix representation of Sect. 3.2 applies to it", which is all Sect. 3 needs for the Boolean decomposition of N . About 4 lines.

GB

2026-09-18: shortest-paths semantics changed from "shortest (to its end vertex)" to the MillenniumDB/GQL one (shortest among the paths satisfying R); Appendix A proves Algorithm 1 for this definition. Notation: Q denotes both the state set of the 2-NFA and the query in $\llbracket Q \rrbracket$; consider $\mathcal{Q}$ for the query.

COMPRESS C12

With the single-destination definitions commented out, this paragraph can be one sentence ("Single-destination variants, defined symmetrically, reduce to single-source ones by reversing the query and the automaton, so we consider only single-source queries"), and the GB note below can go. About 3 lines.

GB

Reviewer D8: the abstract scopes the work to single-source queries while single-destination variants were defined and never evaluated. Kept the definitions and added the reduction remark above. Decide whether to keep the SD definitions or drop them.

3 Linear Algebra, Graphs and Relations

In this section, we establish the linear algebra foundations required for our approach. We introduce the concept of semirings, discuss matrix representations of relations, and present the original LARPQ algorithm for solving the reachability problem.

3.1 Semirings

Semirings provide an algebraic framework that generalizes both Boolean algebra and path-based computations. A semiring consists of a set equipped with two binary operations that satisfy certain axioms. In graph processing algorithms, with respect to GraphBLAS [16], semirings are defined differently from the ones in algebra. The domains of $\otimes$ can vary; distributivity and a multiplicative identity elements are optional.

Definition 3.1 (Semiring). A semiring is a tuple $\langle D_{out}, D_{in_1}, D_{in_2}, \oplus, \otimes, \mathbf{0} \rangle$ where:

- D_{out}, D_{in_1} , and D_{in_2} are three domains¹;
- $\langle D_{out}, \oplus, \mathbf{0} \rangle$ is a commutative monoid with an identity element $\mathbf{0} \in D_{out}$;
- $\otimes : (D_{in_1} \times D_{in_2}) \rightarrow D_{out}$ is a binary operator;

Given a semiring $\mathcal{S} = \langle D_{out}, D_{in_1}, D_{in_2}, \oplus, \otimes, \mathbf{0} \rangle$ we define a *dual semiring* $\overline{\mathcal{S}} = \langle D_{out}, D_{in_2}, D_{in_1}, \oplus, \otimes, \mathbf{0} \rangle$ having $\overline{\otimes}(a, b) = \otimes(b, a)$.

3.2 Matrix Representations

Let $G = \langle V, E, L, \lambda_G \rangle$ be a graph and $N = \langle Q, \Sigma, \Delta, \lambda_N, Q_S, Q_F \rangle$ a 2-NFA. For a label a let $E_a = \{e \in E^{\leftrightarrow} \mid a \in \lambda_G^{\leftrightarrow}(e)\}$ and $\Delta_a = \{\delta \in \Delta \mid a \in \lambda_N(\delta)\}$ be the edges and the transitions labeled by a .

Definition 3.2 (Boolean decomposition). The Boolean decomposition of G is the set of Boolean matrices $\{G^a \mid a \in L^{\leftrightarrow}\}$, where G^a is the $|V| \times |V|$ adjacency matrix of the label a : $G_{uv}^a = 1$ iff $(u, v) \in E_a$. The Boolean decomposition $\{N^a \mid a \in \Sigma^{\leftrightarrow}\}$ of N is defined in the same way over Δ : $N_{qq'}^a = 1$ iff $(q, q') \in \Delta_a$.

3.3 The Path Semiring

For solving path enumeration problems (i.e. to search all simple paths, all trails), we require a more expressive semiring based on path representations.

Let $D_{PATHS} = 2^{\mathbb{N}^*}$, where $\pi' = (v_1, v_2, \dots, v_n) \in \mathbb{N}^*$ represents a 2-path $\pi = ((v_1, v_2), (v_2, v_3), \dots, (v_{n-1}, v_n))$ of length $n - 1$. For $a, b \in D_{PATHS}$ and $c \in \mathbb{B}$, we define the paths semiring and its dual:

- $PATHS = \langle D_{PATHS}, D_{PATHS}, \mathbb{B} = \{0, 1\}, \cup, \text{fst}, \emptyset \rangle$
with $\text{fst}(x, y) = \begin{cases} x & \text{if } y = 1 \\ \emptyset & \text{otherwise} \end{cases}$;
- $\overline{PATHS} = \langle D_{PATHS}, \mathbb{B} = \{0, 1\}, D_{PATHS}, \cup, \text{snd}, \emptyset \rangle$
with $\text{snd}(x, y) = \begin{cases} y & \text{if } x = 1 \\ \emptyset & \text{otherwise} \end{cases}$.

Let $A = (a_{ij})$, $B = (b_{ij})$ be matrices over D_{PATHS} and $C = (c_{ij})$ be a Boolean matrix. We extend the operations to matrices:

- **Sum:** $A_{n \times m} \oplus B_{n \times m} = (a_{ij} \oplus b_{ij})_{n \times m}$;
- **Product:** $A_{n \times m} \otimes C_{m \times k} = (\bigoplus_{l=1}^m a_{il} \otimes c_{lj})_{n \times k}$;
- **Zero-matrix:** $\mathbf{0}_{n \times k}^{PATHS} = (\emptyset)_{n \times k}$.

3.4 Index Unary Operators

¹Actually, types in respective programming language.

The path semiring alone is insufficient for enforcing path semantics (simple paths, trails). We introduce index unary operators that can filter paths based on vertex or edge repetition.

Definition 3.3 (Index unary operator). Let $A_{n \times m} = (a_{ij})$ be a matrix over a semiring R . An index unary operator $I : R \times \mathbb{N} \times \mathbb{N} \mapsto R$ is applied element-wise: $I(A) = (I(a_{ij}, i, j))_{n \times m}$.

Given a Boolean matrix P of size $n \times m$ we define a specific index unary operators for $n \times m$ matrices over $\mathbb{B} = \{0, 1\}$,

$$\bullet I_{reach}^P(a, i, j) = \begin{cases} 1 & \text{if } a = 1 \text{ and } P_{ij} = 0 \\ 0 & \text{otherwise.} \end{cases}$$

Given a D_{PATHS} matrix P of size $n \times m$, for $n \times m$ matrices over D_{PATHS} we define:

$$I_{simple}(a, i, j) = \left\{ (v_1, \dots, v_n, j) \mid \begin{array}{l} (v_1, \dots, v_n) \in a, \\ \forall t \leq n : v_t \neq j \end{array} \right\};$$

$$I_{trail}(a, i, j) = \left\{ (v_1, \dots, v_n, j) \mid \begin{array}{l} (v_1, \dots, v_n) \in a, \\ \forall t < n : (v_t, v_{t+1}) \neq (v_n, j) \end{array} \right\};$$

$$I_{shortest}^P(a, i, j) = \begin{cases} \{(\pi, j) \mid \pi \in a\} & \text{if } P_{ij} = \emptyset, \\ \emptyset & \text{otherwise.} \end{cases}$$

Here (π, j) denotes the vertex sequence π extended by the vertex j . For the shortest-paths semantics we also use the unary operator $\text{minlen}(a) = \{\pi \in a \mid |\pi| \leq |\pi'| \text{ for all } \pi' \in a\}$, which keeps the shortest paths of a set and is applied to the result of the algorithm.

4 A Semiring Framework for Single-Source RPQs

In this section, we present our main contribution: a generalized semiring-based framework for evaluating single-source 2-RPQs.

4.1 Generalized Algorithm

The key insight is that the LARPQ algorithm can be abstracted to work over any semiring equipped with appropriate operations. Algorithm 1 presents the generalized version.

Algorithm 1 Generalized Single-Source 2-RPQ using Semirings

Input: Semiring $\mathcal{S} = \langle D_{out}, D_{in_1}, \mathbb{B}, \oplus, \otimes, \mathbf{0} \rangle$, initial frontier value $\mathbf{1}_{v_s} \in D_{out}$, index unary operator $I^{(\cdot)}$, unary result operator J (the identity unless stated otherwise), 2-NFA $\mathcal{N} = \langle Q, \Sigma, \Delta, \lambda_{\mathcal{N}}, Q_S, Q_F \rangle$, graph $\mathcal{G} = \langle V, E, L, \lambda_{\mathcal{G}} \rangle$, source $v_s \in V$
Output: Result depending on semiring instantiation

- (1) $\{N^a\} \leftarrow$ Boolean decomposition of $\mathcal{N}$
- (2) $\{G^a\} \leftarrow$ Boolean decomposition of $\mathcal{G}$
- (3) $M \leftarrow \mathbf{0}, P \leftarrow \mathbf{0}$
- (4) For each $q \in Q_S$: $M_{qv_s} \leftarrow \mathbf{1}_{v_s}$
- (5) While $M \neq \mathbf{0}$:
 - $P \leftarrow P \oplus M$
 - $M \leftarrow \bigoplus_{a \in \Sigma^{\leftrightarrow} \cap L^{\leftrightarrow}} ((N^a)^T \overline{\otimes} M \otimes G^a)$
 - If I^P defined: $M \leftarrow I^P(M)$
- (6) $F_{1 \times |Q|} \leftarrow \mathbf{0}_{1 \times |Q|}$
- (7) For each $q \in Q_F$: set $F_{1q} \leftarrow 1$
- (8) Return $P^F = J(F \overline{\otimes} P)$

The generalized algorithm differs from the original in several key aspects:

- Matrix operations $(\oplus, \otimes)$ are performed over an arbitrary semiring $\mathcal{S}$ having $D_{in_2} = \mathbb{B}$;

COMPRESS C14

Cut the first two generic sentences; start with "In GraphBLAS [...], semirings are defined differently from the algebraic ones: ...". About 3 lines.

COMPRESS C16

Standard matrix sum/product/zero: one sentence ("Matrix sum, product $\bigoplus_i a_{ij} \otimes c_{ij}$ and the zero matrix are defined as usual, and likewise the dual product $C \overline{\otimes} A$ "). The dual product is used in Algorithm 1 but never defined for matrices. About 3 lines.

COMPRESS C17

Same length, more content: replace "The path semiring alone is insufficient ..." by why an index-aware operator is needed (the multiplicative operator cannot see the column index, so appending the target vertex has to happen in the apply); review-ai.md, M3.

GB

2026-09-18: $I_{shortest}^P$ now tests the pair, $P_{ij} = \emptyset$, instead of the vertex, $\forall k : P_{kj} = \emptyset$, and the result is post-processed by minlen (operator J in Algorithm 1). The vertex test, and likewise a test over the final states only ($\forall k \in Q_F : P_{kj} = \emptyset$), loses the paths that pass through an already answered vertex in another state; see the counterexample in Appendix A. Alternative without minlen : accumulate $F \overline{\otimes} M$ in a separate answer vector inside the loop, masked by the vertices that have no answer yet.

COMPRESS C20

The four bullets restate what Algorithm 1 shows. Two sentences suffice ("Compared with [6], the semiring, the initial frontier value and the post-step operator are parameters; Sect. 4.2 recovers the Boolean case"). About 4 lines. Remove the GB note below once the loop order is confirmed against the implementation.

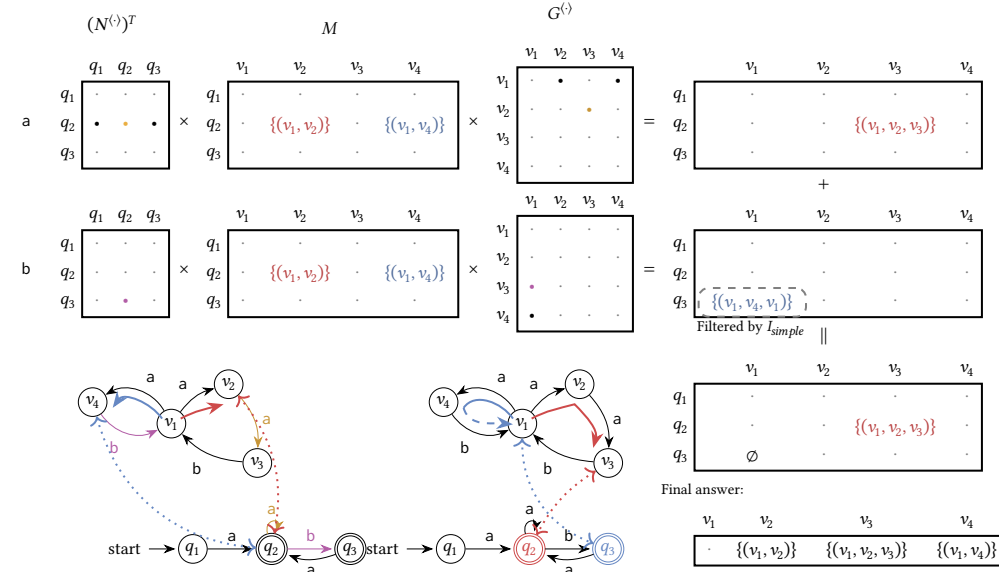


Figure 2: The algorithm step for graph and automaton for regular expression $((a^+)(b?))^+$ using path semiring and all simple paths semantics

- The initial frontier value $\mathbf{1}_{v_s}$ is a parameter of the instantiation, since a semiring in the GraphBLAS sense need not have a multiplicative identity;
- An optional index unary operator I^P is applied to the frontier after each traversal step to filter or transform path information;
- The final result format depends on the semiring instantiation.

We now present two important instantiations of this framework.

4.2 Instantiation I: Boolean Semiring for Reachability

By instantiating $\mathcal{S}$ with the Boolean semiring, i.e., $D_{out} = D_{in_1} = D_{in_2} = \mathbf{B} = \{0, 1\}$, $\oplus = \vee$, $\otimes = \wedge$, and $\mathbf{0} = 0$, the initial frontier value $\mathbf{1}_{v_s} = 1$, and the index unary operator I^P with I_{reach}^P , we recover the original LARPQ algorithm for reachability from [7]. The matrix entries are Boolean values, where 1 indicates reachability. The index unary operator ensures each pair of an automaton state and vertex is processed only once.

The result is a Boolean vector P^F where $P_v^F = 1$ if and only if vertex v is reachable from the source via a path satisfying the regular language constraint.

4.3 Instantiation II: Path Semiring with Index Operators for All Paths

For enumerating all simple paths, all trails, or all shortest paths, we use the previously introduced path semiring $(D_{PATHS}, D_{PATHS}, \mathbf{B} = \{0, 1\}, \cup, \mathbf{fst}, \emptyset)$ combined with appropriate index unary operators. The initial frontier value is the zero-length path from the source, $\mathbf{1}_{v_s} = \{(v_s)\}$.

When $I^P = I_{simple}^P$, the algorithm enumerates all simple 2-paths satisfying the query. When $I^P = I_{trail}^P$, it enumerates all trails. When $I^P = I_{shortest}^P$ and $J = \text{minlen}$, it enumerates all shortest paths. The key difference from the reachability algorithm is that

the frontier matrix M now stores actual path information (as sequences of vertices) rather than Boolean values. The index unary operator filters extensions that would violate the path semantics:

- I_{simple}^P prevents extending a path with a vertex already present in the path (including the starting vertex), ensuring simplicity;
- I_{trail}^P prevents extending a path with an edge already traversed, ensuring the result is a trail;
- $I_{shortest}^P$ prevents extending a path into a pair of a state and a vertex that has already been reached, so only the shortest paths to every such pair are produced; minlen then keeps, for every vertex, the paths of minimal length among those ending in a final state, i.e., the shortest paths that satisfy the constraint.

The algorithm maintains the BFS structure: on iteration n , M_{q_v} contains all valid paths of length n from v_s to v that correspond to paths from some $q_s \in Q_S$ to q in the automaton. Figure 2 illustrates an example of the algorithm step for the all simple paths semantics.

We use **red** and **blue** to denote different paths and the corresponding D_{PATHS} elements in the matrices. Two matrix multiplications correspond to simultaneous graph and automaton traversal steps. Dotted lines represent the relation represented by the matrix M , the relationship between graph vertices and automaton states. The edges affected during the traversal are colored **yellow** for the label a and **purple** for the label b . The corresponding entries in the adjacency matrices use the same colors.

Before the step, the frontier M contains two paths of length 1 in the row of state q_2 : the **red** path $((v_1, v_2))$ in the column of v_2 and the **blue** path $((v_1, v_4))$ in the column of v_4 . The step is performed separately for each label: the product with $(N^a)^T$ and G^a extends the **red** path along the **yellow** edge (v_2, v_3) and the **yellow** self-loop of q_2 , while the product with $(N^b)^T$ and G^b extends the **blue** path along the **purple** edge (v_4, v_1) and the **purple**

GB

Loop body reordered to “accumulate, multiply, filter” (the original text filtered first). With the path semiring, filtering the initial frontier $\{(v_s)\}$ by I_{simple} would append v_s to a path that already contains it and drop everything, so the algorithm would terminate immediately. For the Boolean semiring the new order is exactly the original LARPQ (mask by $\neg P$ after the product). Please double-check against the implementation.

COMPRESS C21

Redundant with Sects. 3.3/3.4; cut this sentence. Keep the three bullets below only if the four descriptive sentences in Sect. 3.4 are cut. About 2 lines.

COMPRESS C22

The colour legend (five sentences) and the step walkthrough below overlap; merge into one paragraph, or move the legend into the caption of Fig. 2. About 3 lines.

transition (q_2, q_3) . For readability, the products in the figure are shown after the new target vertex has been appended to each path, i.e., after applying I_{simple} . After the step, two paths of length 2 are obtained: $\pi_a = ((v_1, v_2), (v_2, v_3))$ and $\pi_b = ((v_1, v_4), (v_4, v_1))$.

The path π_a is simple and is kept, whereas the path π_b is filtered out by the index unary operator I_{simple} because it returns to the already visited vertex v_1 . The per-label results are summed into the new frontier, which is then accumulated into P ; the algorithm finishes with three different paths in the resulting matrix.

The result is a vector P^F over D_{PATHS} where P_v^F contains all 2-paths (all simple paths, all trails, or all shortest paths, depending on the operator) from v_s to v satisfying the regular language constraint.

4.4 Correctness Argument

The correctness argument follows the same induction scheme as in our previous work on LARPQ [7, Appendix A]. The induction is over the iterations of the main loop: the frontier matrix M stores exactly the paths produced at the current traversal depth, and the accumulator matrix P stores the union of all paths produced so far. Each iteration extends paths consistently in the query automaton and in the input graph; the optional index unary operator filters out extensions that violate the selected path semantics. After termination, selecting final automaton states yields exactly the query result for the chosen semiring instantiation. Appendix A gives the proofs for the path-semiring instantiations.

5 Implementation Details

We have implemented the proposed algorithms within the LA-Graph project [21], a collection of linear algebra-based graph processing algorithms built on top of SUITESPARSE:GRAPHBLAS [13]. This section describes the key implementation aspects.

The main engineering limitation is that SUITESPARSE:GRAPH-BLAS user-defined semiring values must have a fixed size to work with built-in automatic allocators. At the same time, an element of the path semiring may contain paths of different lengths and may accumulate an unbounded number of paths during evaluation. To fit these variable-size objects into the fixed-size semiring interface, the implementation uses a compact in-place representation for common cases and falls back to explicit memory management only when this representation overflows.

We introduce two constants, L_{fast} and C_{fast} , to control the maximum path length and the maximum cardinality of a D_{PATHS} element that can be stored without external memory management. As long as all paths in an element are no longer than L_{fast} and the element contains at most C_{fast} paths, the value is stored directly inside the fixed-size semiring object. This fast representation avoids extra allocations and is intended to cover the typical case.

If a path $\pi \in M_{qv}$ with $|\pi| > L_{fast}$ appears, a new linked list representing this long path is allocated in a supplementary memory arena. Similarly, if $|M_{qv}| > C_{fast}$ for some q, v , an extensible linked list is allocated. These fallback structures keep the semiring value itself fixed-size: it stores only the inline data or references to the arena-managed objects. We used $L_{fast} = 8$ and $C_{fast} = 1$, empirically chosen to achieve the best performance in evaluation.

These two values provide a trade-off between peak memory consumption and execution speed. In practice, these constants should be adjusted for each dataset individually.

Finally, we ship the semiring-based algorithm implementation as a standalone executable file. A preprocessed graph, decomposed into a set of Boolean Matrix Market matrices, and a set of regular path query patterns described in a SPARQL-like query language are consumed as input and executed one by one. The CLI flag `--semantics` can be used to switch between all-simple-path, all-shortest-path and all-trail semantics.

6 Evaluation

We evaluate the proposed solution (LARPQ) on three semantics (all-simple-paths, all-shortest-paths, and all-trails) using real-world knowledge graphs. The experiments are designed to answer the following research questions.

- **RQ1:** How does our implementation compare with production graph database systems?
- **RQ2:** What is the performance behavior across different query types and graph structures?

Experiments are conducted on a workstation equipped with a Ryzen 9 7900X (4.7 GHz, 12 cores), 128 GB of DDR5 RAM, running Ubuntu 22.04. The timeout is set to 60 seconds per query. Each query is executed five times: one warm-up run followed by four measurement runs.

Competitors. We compare the performance of the proposed semiring-based LARPQ algorithm against state-of-the-art graph database management systems.

- **MillenniumDB** or MDB (RPQ challenge version) [24]: a persistent, open-source graph database with state-of-the-art RPQ performance and natural support of all three path semantics.
- **FalkorDB** (v4.18.5, formerly RedisGraph [10]): an open-source in-memory property-graph database using Graph-BLAS. Among the three path semantics, only all-trails is naturally expressed in FalkorDB, so we measure only the corresponding queries.

For all competitors, we use their respective query languages to specify queries. The time required for query processing (including parsing and optimization) is included in the measurements. All data is preloaded into storage, so graph preprocessing time is excluded from the results.

Datasets. We used three different graphs and their respective sets of queries.

Wikidata [22]: 610 million edges, 91 million vertices, and 1 400 distinct labels. A total of 578 queries were filtered from a dump of 660 queries.

Yago-2S: 46 million edges, 7 million vertices, and 42 distinct labels. Seven complex queries were taken from [3].

RPQBench [25]: a synthetic dataset with 150 million edges, 57 million vertices, and 9 distinct labels. It includes 20 different query types, 10 queries were generated for each.

6.1 Experimental Results

We measure the total time for all queries, the mean and median time over all queries, and the mean and median per-query speedup over the proposed solution. A summary for all three graphs is presented in Table 1. Per-query results for Wikidata are shown in Figure 3. Only queries that were successfully evaluated for all competitors are included in these statistics. Figure 3d additionally shows queries completed by MillenniumDB and our solution but not by FalkorDB.

COMPRESS C23

DO NOT COMPRESS: Sect. 4.4 should grow, not shrink: proof sketch for simple paths and trails, corrected shortest-paths argument, termination bounds and a cost paragraph (review-ai.md, M1 and M3). Budget about 1 page.

COMPRESS C24

"Inline when small, arena otherwise" is said three times in this section (this sentence, "This fast representation avoids ...", "These fallback structures keep ..."). Keep the sentence with the constants and the two fallback sentences. About 4 lines.

COMPRESS C25

This sentence and "should be adjusted for each dataset" contradict the single setting that was used. Replace both by a two-line sensitivity result (ADBIS D5) or drop them. About 2 lines.

COMPRESS C26

The CLI description (three sentences) can be a footnote, or one sentence next to the artifact link in Sect. 8. About 3 lines.

COMPRESS C27

Whole section: the analysis prose after Table 1 (six paragraphs) is long and thin. Replace it by (i) a small table with completed/timed-out queries per system and semantics and the n of each column group, (ii) two or three paragraphs that state facts with numbers, (iii) a per-query YAGO-2S table. Net length about the same, but it answers ADBIS D1/D3/D5 and review-ai.md, M4.

COMPRESS C29

Generic; cut. The informal simple/hard definition in the next sentence should go too (use "above/below the median" or a threshold). About 3 lines.

Table 1: Performance summary on Wikidata, RPQBench, and YAGO-2S (time in milliseconds). For YAGO-2S under the simple-paths and trails semantics only two of seven queries completed on all systems, so only total time is reported; FalkorDB timed out on all YAGO-2S queries (—).

Dataset	Metric	Simple		Shortest		Trails		
		LARPQ	MDB	LARPQ	MDB	LARPQ	MDB	FalkorDB
Wikidata	Total time	224 682	1 111 429	205 629	780 011	152 128	921 506	81 519
	Mean time	428	2 121	392	1 488	312	1 892	167
	Median time	1.7	36.6	2.0	27.9	1.4	36.9	4.5
	Mean speedup	1.0	0.6	1.0	0.8	1.0	0.5	0.9
	Median speedup	1.0	0.1	1.0	0.1	1.0	0.1	0.7
RPQBench	Total time	857	2 836	890	3 253	849	3 256	1 685
	Mean time	43	142	45	162	53	203	105
	Median time	0.3	0.2	0.3	0.1	0.3	0.2	0.2
	Mean speedup	1.0	1.5	1.0	2.1	1.0	1.5	1.2
	Median speedup	1.0	1.8	1.0	2.4	1.0	1.7	1.5
YAGO-2S	Total time	105	215	1 195	1 956	66	479	—
	Mean time	—	—	171	279	—	—	—
	Median time	—	—	167	268	—	—	—
	Mean speedup	—	—	1.0	0.5	—	—	—
	Median speedup	—	—	1.0	0.6	—	—	—

The evaluation shows that execution time strongly depends on the particular query and the choice of start and target vertices. We define a query as *simple* for a system if it requires relatively little time to evaluate, and *hard* if, conversely, it requires a relatively large amount of time.

For RPQBench (Table 1), the total and mean execution times of our solution are more than three times better than those of the competitors across all semantics. However, the median time and per-query speedup favor the competitors. This indicates that the dataset contains a relatively large number of simple queries, while our solution exhibits a significant advantage on harder queries.

This observation is consistent with the results presented in Figure 3, where MillenniumDB outperforms our solution on simple queries but is slower on hard ones. The tail of queries that are challenging for all solutions is thinner for LARPQ than for MillenniumDB and FalkorDB, as illustrated in Figure 3c. Additionally, we observe that in the common-trails setting, FalkorDB achieves the best total and mean times, while LARPQ achieves the best median time, which is also consistent with the shape of the hard queries tail.

Analysis of the YAGO-2S dataset (see Table 1) shows that the shortest-path semantics is the only one for which all seven queries completed successfully across all competing systems. For the simple-paths and trails semantics, only two of the seven queries were completed by all systems within the 60-second timeout; the remaining queries timed out on every solution, and FalkorDB timed out on all YAGO-2S queries. Therefore, for these two semantics, Table 1 reports only the total time over the two completed queries and omits the mean, median, and speedup, which are not meaningful for such a small sample. For the queries that did complete, our solution is faster than the competitors, but this observation is based on two queries only.

A detailed analysis of which queries are *simple* and which are *hard* for our solution yields results similar to those reported in [7], indicating that the proposed generalization significantly inherits the properties of the basic version. Specifically, LARPQ outperforms competitors on queries with complex patterns over labels corresponding to many edges (e.g., $l_1 \cdot l_2 \cdot (l_3 \mid l_4)^*$), while

on queries with simple patterns such as $l_1^* \cdot l_2$, it shows no performance advantage, and MillenniumDB may be a better choice.

Overall, the results indicate that LARPQ is a strong choice for complex RPQs, especially when query patterns involve labels with many matching edges, whereas competing systems may perform better on simpler patterns.

7 Related Work

The algebraic approach to path problems has a long history. Kleene’s algorithm computes reachability using matrix operations over the Boolean semiring. This approach has been generalized to arbitrary semirings for solving various algebraic path problems [5], including path enumeration. Different query semantics and respective algorithms for regular path queries, including all shortest paths and all simple paths, have been studied in the literature [18, 19, 23].

The GraphBLAS initiative [9, 13, 16] established a standardized interface for expressing graph algorithms using linear algebra operations. The GraphBLAS specification supports arbitrary semirings, enabling the expression of algorithms that go beyond traditional numeric computations. LAGraph [21] provides a collection of graph algorithms built on GraphBLAS, including BFS, PageRank, and triangle counting. Recent work has explored linear algebra approaches for various graph problems. However, RPQ evaluation in this paradigm remains underexplored, with only a few prior works [4, 7] addressing this problem.

Several recent results are closely related to ours but target different settings. cuRPQ [20] accelerates RPQ and conjunctive RPQ (CRPQ) evaluation on GPUs with a dedicated traversal algorithm and visited-set management; we instead build on general-purpose sparse linear algebra kernels on CPUs, and offloading the same matrix operations to GPUs is a natural next step for our approach. Abo Khamis et al. [2] give output-sensitive algorithms for acyclic CRPQs whose complexity matches that of the corresponding conjunctive queries; this is a theoretical result at the level of CRPQs under reachability semantics, whereas we focus on the practical single-source evaluation of 2-RPQs under path semantics. LD-RPQB [17] is a synthetic RPQ benchmark whose data graph follows prescribed path-length distributions; it

COMPRESS C30
Also inaccurate for Wikidata, where LARPQ wins the median as well (1.7 vs 36.6 ms). Merge this and the next two sentences into one paragraph on the shape of the distributions. About 2 lines.

COMPRESS C31
Four sentences for two numbers. With a per-query YAGO-2S table (7 rows $\times$ 3 semantics) this is one sentence, and the “—” cells in Table 1 can go. About 3 lines.

COMPRESS C32
Unsupported here (no per-type data is shown): either add a per-type RPQBench figure, which would also answer RQ2, or reduce to one sentence citing [6]. The “Overall, ...” sentence below repeats it; keep one of the two. About 4 lines.

COMPRESS C33
Fig. 3 takes about 0.4 page for four panels whose tick labels are unreadable. One panel with grouped violins (system $\times$ semantics) on a shared log axis, or a CDF of per-query time ratios, is smaller and directly comparable. About 0.2 page.

COMPRESS C34
Compress? This is already the shortest section (about 0.35 page) and both ADBIS reviewers asked to *strengthen* it (D6, R4). Cut the filler marked below, but spend the saved lines on the missing references: classical path algebras (Carré, Lehmann, Gondran–Minoux, Mohri), provenance semirings, RPQ engines with GQL path modes, and the group’s own CFPQ-in-linear-algebra work (review-ai.md, M6). Net length: unchanged or slightly longer.

COMPRESS C35
Three sentences on GraphBLAS/LAGraph; one is enough (“GraphBLAS [...] standardizes graph algorithms over arbitrary semirings; LAGraph [...] collects algorithms built on it”). Also cut “Recent work has explored ...” below (vague, uncited). About 3 lines.

COMPRESS C36
Each of the three positioning sentences can lose its second clause (about 30%). About 2 lines.

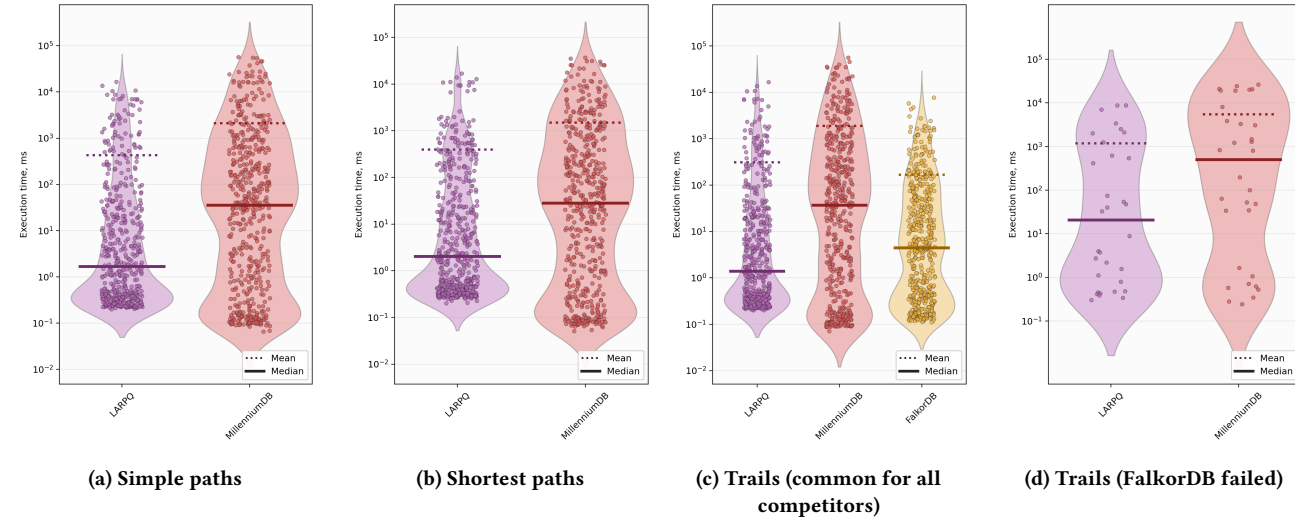


Figure 3: Per-query execution time on Wikidata (log scale). Each point is one query; solid and dotted lines are the median and the mean.

complements RPQBench [25], which we use, and is a candidate for extending our evaluation.

8 Conclusion and Future Work

We have presented a semiring-based generalization of the linear algebra approach for evaluating two-way regular path queries. Source code and additional materials are available at GitHub². Experimental evaluation demonstrates that the proposed approach achieves competitive performance with state-of-the-art graph database systems while offering a more flexible framework for different path semantics. At the same time, our evaluation reveals limitations of both the proposed and existing solutions in terms of performance (see the discussion of results on YAGO-2S and Table 1), opening the door for further research on performance improvements.

Future work includes the use of additional semirings, such as counting semirings for path counting, as well as integration with property graphs and more expressive query languages such as CRPQs. Another direction is distributed implementation and acceleration through offloading matrix operations to GPUs.

Acknowledgments

This research has been supported by the St. Petersburg State University, grant number 116636233.

References

- [1] 2013. *SPARQL 1.1 Query Language*. Technical Report. W3C. <http://www.w3.org/TR/sparql11-query>
- [2] Mahmoud Abo Khamis, Alexandru-Mihai Hurjui, Ahmet Kara, Dan Olteanu, and Dan Suciu. 2026. Acyclic Conjunctive Regular Path Queries are no Harder than Corresponding Conjunctive Queries. *Proceedings of the ACM on Management of Data* 4, 2 (PODS) (2026), 1–24. doi:10.1145/3801891
- [3] Zahid Abul-Basher, Nikolay Yakovets, Parke Godfrey, Shadi Ghajar-Khosravi, and Mark H. Chignell. 2017. TASWEET: Optimizing Disjunctive Path Queries in Graph Databases. In *Proceedings of the 20th International Conference on Extending Database Technology, EDBT 2017, Venice, Italy, March 21-24, 2017*, Volker Markl, Salvatore Orlando, Bernhard Mitschang, Periklis Andritsos, Kai-Uwe Sattler, and Sebastian Breß (Eds.). OpenProceedings.org, 470–473. doi:10.5441/002/EDBT.2017.47
- [4] Diego Arroyuelo, Adrián Gómez-Brandón, and Gonzalo Navarro. 2023. Evaluating Regular Path Queries on Compressed Adjacency Matrices. In *String Processing and Information Retrieval: 30th International Symposium, SPIRE 2023*,

²Generalized LARPO: <https://github.com/SparseLinearAlgebra/LAGraph/pull/13>

- Pisa, Italy, September 26–28, 2023, Proceedings* (Pisa, Italy). Springer-Verlag, Berlin, Heidelberg, 35–48. doi:10.1007/978-3-031-43980-3_4
- [5] John S. Baras and George Theodorakopoulos. 2010. Path Problems in Networks. *Synthesis Lectures on Communication Networks* 3 (2010), 1–77. <https://api.semanticscholar.org/CorpusID:38871641>
- [6] Chris Barrett, Riko Jacob, and Madhav Marathe. 2000. Formal-Language-Constrained Path Problems. *SIAM J. Comput.* 30, 3 (2000), 809–837. doi:10.1137/S0097539798337716
- [7] Georgiy Belyanin, Rodion Suvorov, and Semyon Grigorev. 2025. Single-Source Regular Path Querying in Terms of Linear Algebra. In *Proceedings of the VLDB 2025 Workshops: Large Scale Graph Data Analytics (LSGDA)*. London, United Kingdom. https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/LSGDA/LSGDA25_12.pdf Extended version: arXiv:2412.10287.
- [8] Meghyn Bienvenu, Diego Calvanese, Magdalena Ortiz, and Mantas Simkus. 2014. Nested Regular Path Queries in Description Logics (Extended Abstract). In *Proceedings of the 8th Alberto Mendelzon Workshop on Foundations of Data Management, Cartagena de Indias, Colombia, June 4-6, 2014 (CEUR Workshop Proceedings)*, Georg Gottlob and Jorge Pérez (Eds.). CEUR-WS.org. https://ceur-ws.org/Vol-1189/paper_34.pdf
- [9] Benjamin Brock, Aydın Buluç, Timothy Mattson, Scott McMillan, and José Moreira. 2021. The graphblas c api specification. *GraphBLAS.org, Tech. Rep* (2021).
- [10] Pieter Cailliau, Tim Davis, Vijay Gadepally, Jeremy Kepner, Roi Lipman, Jeffrey Lovitz, and Keren Ouaknine. 2019. RedisGraph GraphBLAS Enabled Graph Database. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 285–286. doi:10.1109/ipdpsw.2019.00054
- [11] Diego Calvanese, Giuseppe De Giacomo, Maurizio Lenzerini, and Moshe Y. Vardi. 2003. Reasoning on Regular Path Queries. *SIGMOD Record* 32, 4 (2003), 83–92. doi:10.1145/959060.959076
- [12] Katrin Casel and Markus L. Schmid. 2021. Fine-Grained Complexity of Regular Path Queries. In *24th International Conference on Database Theory (ICDT 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 186)*, Ke Yi and Zhewei Wei (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 19:1–19:20. doi:10.4230/LIPIcs.ICDT.2021.19
- [13] Timothy A. Davis. 2023. Algorithm 1037: SuiteSparse:GraphBLAS: Parallel Graph Algorithms in the Language of Sparse Linear Algebra. *ACM Trans. Math. Softw.* 49, 3, Article 28 (Sept. 2023), 30 pages. doi:10.1145/3577195
- [14] Benjamin Farias, Carlos Rojas, and Domagoj Vrgoc. 2023. MillenniumDB path query challenge (short paper). In *Proceedings of the 15th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW 2023), Santiago de Chile, Chile, May 22-26, 2023 (CEUR Workshop Proceedings)*, Benny Kimelfeld, Maria Vanina Martinez, and Renzo Angles (Eds.). CEUR-WS.org. <https://ceur-ws.org/Vol-3409/paper13.pdf>
- [15] ISO/IEC 39075:2024 2024. *Information technology – Database languages – GQL*. Standard. International Organization for Standardization, Geneva, CH. <https://www.iso.org/standard/76120.html>
- [16] Jeremy Kepner, Peter Aaltonen, David A. Bader, Aydın Buluç, Franz Franchetti, John R. Gilbert, Dylan Hutchison, Manoj Kumar, Andrew Lumsdaine, Henning Meyerhenke, Scott McMillan, Carl Yang, John Douglas Owens, Marcin Zalewski, Timothy G. Mattson, and José E. Moreira. 2016. Mathematical foundations of the GraphBLAS. *2016 IEEE High Performance Extreme Computing Conference (HPEC)* (2016), 1–9. <https://api.semanticscholar.org/CorpusID:3654505>

COMPRESS C37

Repeats the abstract; merge with the next sentence ("Experiments show competitive performance ..., but also limitations of all systems on YAGO-2S ..."). About 2 lines.

- [17] Menglu Ma, Hui Wang, Xin Wang, Yiheng You, and Jiake Ge. 2025. LD-RPQB: A Benchmark for Regular Path Queries Based on Length Distribution. *World Wide Web* 28, 5 (2025), 52. doi:10.1007/s11280-025-01365-9
- [18] Wim Martens and Tina Trautner. 2018. Evaluation and Enumeration Problems for Regular Path Queries. In *21st International Conference on Database Theory (ICDT 2018) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 98)*, Benny Kimelfeld and Yael Amsterdamer (Eds.), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 19:1–19:21. doi:10.4230/LIPIcs.ICDT.2018.19
- [19] Alberto O. Mendelzon and Peter T. Wood. 1989. Finding Regular Simple Paths in Graph Databases. *SIAM J. Comput.* 24 (1989), 1235–1258. https://api.semanticscholar.org/CorpusID:12684556
- [20] Sungwoo Park, Seohyeon Kim, and Min-Soo Kim. 2026. cuRPQ: A High-Performance GPU-Based Framework for Processing Regular and Conjunctive Regular Path Queries. *Proceedings of the ACM on Management of Data* 4, 3 (SIGMOD) (2026), 1–28. doi:10.1145/3802033
- [21] Gábor Szárnyas, David A. Bader, Timothy A. Davis, James Kitchen, Timothy G. Mattson, Scott McMillan, and Erik Welch. 2021. LAGraph: Linear Algebra, Network Analysis Libraries, and the Study of Graph Algorithms. arXiv:2104.01661 [cs.MS] https://arxiv.org/abs/2104.01661
- [22] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Commun. ACM* 57, 10 (sep 2014), 78–85. doi:10.1145/2629489
- [23] Domagoj Vrgoc. 2022. Evaluating regular path queries under the all-shortest paths semantics. *CoRR abs/2204.11137* (2022). doi:10.48550/ARXIV.2204.11137 arXiv:2204.11137
- [24] Domagoj Vrgoc, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. 2021. MillenniumDB: A Persistent, Open-Source, Graph Database. arXiv:2111.01540 [cs.DB] https://arxiv.org/abs/2111.01540
- [25] Hui Wang, Xin Wang, Menglu Ma, and Yiheng You. 2025. RPQBench: A Benchmark for Regular Path Queries on Graph Data. In *Web Information Systems Engineering – WISE 2024*, Mahmoud Barhamgi, Hua Wang, and Xin Wang (Eds.). Springer Nature Singapore, Singapore, 351–367.

A Correctness of Algorithm 1

Throughout, $\langle G, R, v_s \rangle$ is a 2-RPQ and N is a 2-NFA accepting R , with the components of Sect. 2: a word $a_1 \dots a_n$ belongs to R iff there is a path $(q_0, \dots, q_n)$ in N with $q_0 \in Q_S$, $q_n \in Q_F$, and $a_i \in \lambda_N((q_{i-1}, q_i))$ for all i . As in Sect. 3, a 2-path is identified with its vertex sequence; we index it from zero, $\pi = (v_0, \dots, v_n)$, so that $|\pi| = n$, and write $\pi \cdot u$ for $(v_0, \dots, v_n, u)$. For a matrix M we write $M[q, v]$ for its entry in row q and column v . We call a 2-path π from v_s *accepted* if $\pi \in \mathcal{A}$, i.e., $\omega_{G^{\leftrightarrow}}(\pi) \cap R \neq \emptyset$.

Runs and witnesses. A *run* of N on a 2-path $\pi = (v_0, \dots, v_n)$ is a path $\rho = (q_0, \dots, q_n)$ in N with $q_0 \in Q_S$ and $\lambda_G^{\leftrightarrow}((v_{i-1}, v_i)) \cap \lambda_N((q_{i-1}, q_i)) \neq \emptyset$ for all $1 \leq i \leq n$. Choosing one label in each of these intersections, and conversely reading off the states of an accepting path of N on a word of $\omega_{G^{\leftrightarrow}}(\pi)$, shows that

$$\pi \text{ is accepted} \iff \text{some run of } N \text{ on } \pi \text{ ends in } Q_F. \quad (1)$$

For $n \geq 0$, $q \in Q$, and $v \in V$ let

$$\mathcal{W}_n(q, v) = \{\pi \mid \pi \text{ is a 2-path of length } n \text{ from } v_s \text{ to } v \text{ with a run ending in } q\}$$

be the set of *witnesses* of (q, v) at depth n , and let $d(q, v) = \min\{n \mid \mathcal{W}_n(q, v) \neq \emptyset\}$, with $\min \emptyset = \infty$.

States of the algorithm. Let M_n and P_n denote the matrices M and P after n iterations of the loop of Algorithm 1; M_0 is the initial frontier and $P_0 = \mathbf{0}$. Writing

$$\text{step}(M) = \bigoplus_{a \in \Sigma^{\leftrightarrow} \cap L^{\leftrightarrow}} (N^a)^T \bar{\otimes} M \otimes G^a$$

for the traversal step, the loop body gives, for $n \geq 1$,

$$P_n = P_{n-1} \oplus M_{n-1} = \bigoplus_{k < n} M_k \quad \text{and} \quad M_n = I^{P_n}(\text{step}(M_{n-1})).$$

If the loop stops after T iterations, then $M_T = \mathbf{0}$ and the returned accumulator is $P_T = \bigoplus_k M_k$. For the path semiring, $\oplus$ is the entrywise union, the initial frontier is $M_0[q, v] = \{(v_s)\}$ if $q \in Q_S$ and $v = v_s$ and $\emptyset$ otherwise, and the index unary operator is

always applied, since it performs the extension of the paths by the target vertex.

LEMMA A.1. *For every $Q \times V$ matrix M over D_{PATHS} and all $q \in Q$, $u \in V$,*

$$\text{step}(M)[q, u] = \bigcup \{M[q', v] \mid (q', q) \in \Delta_a \text{ and } (v, u) \in E_a \text{ for some } a \in \Sigma^{\leftrightarrow} \cap L^{\leftrightarrow}\}.$$

PROOF. By the definition of the matrix operations over $PATHS$ and $\bar{PATHS}$,

$$\begin{aligned} ((N^a)^T \bar{\otimes} M)[q, v] &= \bigcup_{q'} \text{snd}(N^a[q', q], M[q', v]) \\ &= \bigcup_{(q', q) \in \Delta_a} M[q', v], \\ (X \otimes G^a)[q, u] &= \bigcup_v \text{fst}(X[q, v], G^a[v, u]) \\ &= \bigcup_{(v, u) \in E_a} X[q, v]; \end{aligned}$$

summing over a gives the claim. $\square$

LEMMA A.2. *Let $M[q, v] \subseteq \mathcal{W}_n(q, v)$ for all q, v , and let $M' = \text{step}(M)$. Then for all $q \in Q$ and $u \in V$:*

- (i) *if $\pi \in M'[q, u]$, then $\pi \cdot u \in \mathcal{W}_{n+1}(q, u)$;*
- (ii) *if $\pi \cdot u \in \mathcal{W}_{n+1}(q, u)$ has a run $(q_0, \dots, q_n, q)$ and $\pi \in M[q_n, v_n]$, where v_n is the last vertex of π , then $\pi \in M'[q, u]$.*

PROOF. (i) By Lemma A.1, $\pi \in M[q', v] \subseteq \mathcal{W}_n(q', v)$ for some q', v , and a with $(q', q) \in \Delta_a$ and $(v, u) \in E_a$. Hence π ends in v , $\pi \cdot u$ is a 2-path of length $n + 1$ from v_s to u , and appending q to a run on π that ends in q' yields a run on $\pi \cdot u$, because $a \in \lambda_G^{\leftrightarrow}((v, u)) \cap \lambda_N((q', q))$. (ii) The run provides a label $a \in \lambda_G^{\leftrightarrow}((v_n, u)) \cap \lambda_N((q_n, q))$, i.e., $(q_n, q) \in \Delta_a$ and $(v_n, u) \in E_a$, so $M[q_n, v_n] \subseteq M'[q, u]$ by Lemma A.1. $\square$

THEOREM A.3 (SIMPLE PATHS AND TRAILS). *Let X be simple or trail, let $I^P = I_X$, and let $\mathcal{W}_n^X(q, v)$ be the set of paths in $\mathcal{W}_n(q, v)$ that are X . Then, for all $n \geq 0$, $q \in Q$, and $v \in V$,*

$$M_n[q, v] = \mathcal{W}_n^X(q, v). \quad (2)$$

Consequently, the loop terminates after at most $|V|$ iterations for simple paths and at most $|E^{\leftrightarrow}| + 1$ iterations for trails, and $P^F[u] = \{\pi \in \llbracket Q \rrbracket_X \mid \pi \text{ ends in } u\}$ for every $u \in V$.

PROOF. We use two properties of X -paths: a prefix of an X -path is an X -path, and $\pi \cdot u$ is an X -path iff π is an X -path and u does not occur in π (for $X = \text{simple}$) or (v_n, u) is not an edge of π (for $X = \text{trail}$). The second property says that, for an X -path π ending in v_n , the operator I_X keeps $\pi \cdot u$ iff $\pi \cdot u$ is an X -path.

We prove (2) by induction on n . For $n = 0$ both sides equal $\{(v_s)\}$ if $q \in Q_S$ and $v = v_s$, and $\emptyset$ otherwise: the zero-length path is simple and a trail, and its runs are the one-state paths (q) with $q \in Q_S$. Assume (2) for n and let $M' = \text{step}(M_n)$, so that $M_{n+1}[q, u] = I_X(M'[q, u], q, u)$. ($\subseteq$) Let $\pi \cdot u \in M_{n+1}[q, u]$. Then $\pi \in M'[q, u]$, so $\pi \cdot u \in \mathcal{W}_{n+1}(q, u)$ by Lemma A.2(i); moreover $\pi \in M_n[q', v]$ for some q', v by Lemma A.1, so π is an X -path by the hypothesis, and $\pi \cdot u$ is an X -path because I_X kept it. ($\supseteq$) Let $\pi \cdot u \in \mathcal{W}_{n+1}^X(q, u)$ with a run $(q_0, \dots, q_n, q)$. Its prefix π is an X -path of length n from v_s to v_n with the run $(q_0, \dots, q_n)$, hence $\pi \in \mathcal{W}_n^X(q_n, v_n) = M_n[q_n, v_n]$. By Lemma A.2(ii), $\pi \in M'[q, u]$, and I_X keeps $\pi \cdot u$ because it is an X -path.

Termination: a simple path visits every vertex at most once, so $\mathcal{W}_n^{\text{simple}} = \emptyset$ for $n \geq |V|$; a trail uses every edge of $E^{\leftrightarrow}$ at most once, so $\mathcal{W}_n^{\text{trail}} = \emptyset$ for $n > |E^{\leftrightarrow}|$. By (2), $M_n = \mathbf{0}$ from these n on. Result: $P = \bigoplus_n M_n$ and $(F \bar{\otimes} P)[u] = \bigcup_q \text{snd}(F[q], P[q, u]) =$

$\bigcup_{q \in Q_F} \bigcup_n \mathcal{W}_n^X(q, u)$ is the set of X -2-paths from v_s to u having a run that ends in Q_F , i.e., by (1), the set of paths of $\llbracket Q \rrbracket_X$ that end in u . $\square$

THEOREM A.4 (SHORTEST PATHS). *Let I^P be the operator*

$$I_{shortest}^P(a, q, u) = \begin{cases} \{\pi \cdot u \mid \pi \in a\} & \text{if } P[q, u] = \emptyset, \\ \emptyset & \text{otherwise,} \end{cases}$$

and let $J = \text{minlen}$, so that $P^F = \text{minlen}(F \otimes P)$ entrywise. Then, for all $n \geq 0$, $q \in Q$, and $v \in V$,

$$M_n[q, v] = \begin{cases} \mathcal{W}_n(q, v) & \text{if } d(q, v) = n, \\ \emptyset & \text{otherwise.} \end{cases} \quad (3)$$

Consequently, the loop terminates after at most $|Q| \cdot |V|$ iterations, and $P^F[u] = \{\pi \in \llbracket Q \rrbracket_{shortest} \mid \pi \text{ ends in } u\}$ for every $u \in V$.

PROOF. We prove (3) by induction on n . The case $n = 0$ is as in Theorem A.3, since $d(q, v_s) = 0$ iff $q \in Q_s$, and $d(q, v) \geq 1$ for $v \neq v_s$. Assume (3) for all $k \leq n$. Then $P_{n+1}[q, u] = \bigcup_{k \leq n} M_k[q, u]$ is non-empty iff $d(q, u) \leq n$. Hence $M_{n+1}[q, u] = \emptyset$ if $d(q, u) \leq n$, as required, and otherwise $M_{n+1}[q, u] = \{\pi \cdot u \mid \pi \in M'[q, u]\}$ with $M' = \text{step}(M_n)$; note that $M_n[q, v] \subseteq \mathcal{W}_n(q, v)$ for all q, v , so Lemma A.2 applies. Let $d(q, u) \geq n + 1$. ($\subseteq$) If $\pi \in M'[q, u]$, then $\pi \cdot u \in \mathcal{W}_{n+1}(q, u)$ by Lemma A.2(i); hence $d(q, u) = n + 1$, and $\pi \cdot u$ belongs to the right-hand side of (3). ($\supseteq$) Let $d(q, u) = n + 1$ and $\pi \cdot u \in \mathcal{W}_{n+1}(q, u)$ with a run $(q_0, \dots, q_n, q)$. Then $\pi \in \mathcal{W}_n(q_n, v_n)$, so $d(q_n, v_n) \leq n$. In fact $d(q_n, v_n) = n$: a witness $\pi'' \in \mathcal{W}_m(q_n, v_n)$ with $m < n$ would give, by the argument of Lemma A.2(i), $\pi'' \cdot u \in \mathcal{W}_{m+1}(q, u)$ with $m + 1 < d(q, u)$. Hence $\pi \in M_n[q_n, v_n]$ by the hypothesis, $\pi \in M'[q, u]$ by Lemma A.2(ii), and $\pi \cdot u \in M_{n+1}[q, u]$.

Termination: a witness of minimal length, together with its run, never visits a pair (q_i, v_i) twice, since the segment between two visits could be removed; hence $d(q, v) < |Q| \cdot |V|$ whenever it is finite, and $M_n = \mathbf{0}$ for $n \geq |Q| \cdot |V|$ by (3). Result: by (3), $A_u = (F \otimes P)[u] = \bigcup_{q \in Q_F} \mathcal{W}_{d(q, u)}(q, u)$, where the terms with $d(q, u) = \infty$ are empty, is the set of 2-paths π from v_s to u that have a run ending in some $q \in Q_F$ with $|\pi| = d(q, u)$. Let $d_F(u) = \min_{q \in Q_F} d(q, u)$; then $\text{minlen}(A_u)$ consists of the paths of A_u of length $d_F(u)$, and each of them is accepted by (1). Every accepted path π' from v_s to u has a run ending in some $q' \in Q_F$, so $|\pi'| \geq d(q', u) \geq d_F(u)$; therefore the paths of $\text{minlen}(A_u)$ belong to $\llbracket Q \rrbracket_{shortest}$. Conversely, let $\pi \in \llbracket Q \rrbracket_{shortest}$ end in u , with a run ending in $q \in Q_F$. Since $\mathcal{W}_{d_F(u)}(q^*, u)$ is non-empty for the minimizing $q^* \in Q_F$ and consists of accepted paths, the minimality of π gives $|\pi| \leq d_F(u) \leq d(q, u) \leq |\pi|$, so $|\pi| = d(q, u) = d_F(u)$, $\pi \in \mathcal{W}_{d(q, u)}(q, u) \subseteq A_u$, and $\pi \in \text{minlen}(A_u)$. $\square$

Remark A.5. Replacing every set in (3) by its non-emptiness gives $M_n[q, v] = 1$ iff $d(q, v) = n$, the invariant of the Boolean instantiation of Sect. 4 and the correctness statement of [7]; the same induction proves it, with Lemma A.1 read over the Boolean semiring and the operator I_{reach}^P instead of $I_{shortest}^P$.

GB

Sects. 2 and 3 and Algorithm 1 were aligned with this theorem on 2026-09-18. The previous vertex-level test $\forall k : P[k, u] = \emptyset$, and likewise a test over the final states only, make the statement false: for $R = a \mid bcd$ on the graph $s \xrightarrow{a} u, s \xrightarrow{b} x, x \xrightarrow{c} u, u \xrightarrow{d} w$, the vertex u is accepted at depth 1, the entry into u at depth 2 is filtered, and the only accepted path s, x, u, w to w is never produced.

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007

1008